Do this includes Server-Side Request Forgery and Role-Based Access Control?

Yes—both should be included, but they belong in different layers of the guardrails package.

- **SSRF** is primarily an **input/output and tool/network guardrail**: it prevents user prompts, retrieved documents, or model outputs from causing the server to fetch internal or unauthorized resources.
- **RBAC** is primarily a **retrieval authorization guardrail**: it ensures a user can only retrieve chunks, documents, indexes, or tools permitted for their role, tenant, and task scope.

OWASP specifically flags SSRF as an LLM-application risk when an attacker causes unintended requests to internal services, APIs, or data stores, and recommends strict validation, allowlists, sandboxing, and monitoring. For RAG, access control should be enforced at retrieval time—not only during ingestion—using server-side identity, role, tenant, classification, and task scope. [1] [2] [3] [4]

Where they fit

Guardrail	Stage	What it does	Example rule
SSRF prevention	Input, retrieval, tool-use, output	Validates URLs, domains, IPs, protocols, ports, and redirects before any server-side fetch	Block requests to `localhost`, private IP ranges, cloud metadata endpoints, and non-allowlisted domains
RBAC / authorization	Retrieval and tool-use	Filters retrieval results and tool actions by authenticated user role and permissions	A `support` user cannot retrieve HR payroll documents; a `finance` user can

SSRF guardrails to add

- URL/schema validation: allow only approved protocols such as HTTPS.
- Domain and hostname allowlists; deny-by-default policy.
- Block private, loopback, link-local, and cloud-metadata IP ranges.
- Resolve DNS and validate the resolved IP before making the request.
- Disable or tightly control redirects; revalidate each redirect target.
- Restrict ports, request methods, headers, response size, and timeout.
- Scan retrieved documents and model output for embedded URLs before any downstream fetch.
- Log and alert on blocked internal-network access attempts.

OWASP advises positive allowlists for URL schema, port, and destination, and warns that denylists or regexes alone are insufficient for SSRF mitigation. [5]

RBAC guardrails to add

- Identity context: user ID, roles, groups, tenant ID, clearance, and task scope.
- Document/chunk metadata policies: department, confidentiality, owner, region, and retention status.
- Query-time filtering: apply authorization filters before ranking or immediately after retrieval.
- Post-retrieval authorization re-check for every candidate chunk.
- Tool/action authorization: separate permissions for search, file access, database query, email, or external API calls.
- Tenant isolation and cross-tenant leakage prevention.
- Audit logging of allowed and denied retrieval or tool actions.

A RAG system should not return content merely because it is semantically similar to the question; authorization must be evaluated server-side for the caller, service principal, and current task. [3]

Updated package scope

Add two new modules:

- `network/` — SSRF, URL validation, egress controls, redirect handling, and private-network blocking.
- `authz/` — RBAC, ABAC-style attribute checks, tenant isolation, retrieval-time authorization, and tool permission enforcement.

So the prior plan should be expanded from “input, retrieval, generation, output” to include **network egress** and **authorization** as first-class guardrail stages.

⁂

1. https://owasp.github.io/www-project-top-10-for-large-language-model-applications/Archive/0_1_vulns/SSRF.html
2. https://cheatsheetseries.owasp.org/cheatsheets/Server_Side_Request_Forgery_Prevention_Cheat_Sheet.html
3. https://github.com/OWASP/AISVS/blob/main/1.01/research/chapters/C08-Memory-and-Embeddings/C08-01-Access-Controls-Memory-RAG.md
4. https://github.com/gh4rib/OwaspCheatSheetSeries/blob/master/cheatsheets/RAG_Security_Cheat_Sheet.md
5. https://github.com/OWASP/Top10/blob/master/2021/docs/en/A10_2021-Server-Side_Request_Forgery_(SSRF).md
6. https://pangea.cloud/blog/ai-access-granted-rag-apps-with-identity-and-access-control/
7. https://github.com/OWASP/AISVS/blob/8906b2be8341abbf4dc43411374681fb81e6ddc6/1.0/research/chapters/C08-Memory-and-Embeddings/C08-01-Access-Controls-Memory-RAG.md
8. https://github.com/microsoft/hve-core/blob/main/.github/skills/security/owasp-llm/references/05-improper-output-handling.md

9. https://github.com/OWASP/Top10/blob/master/osib/docs/A10_2021-Server-Side_Request_Forgery_(SSRF).md
10. https://github.com/jonesemjay/OWASP-CheatSheetSeries/blob/master/cheatsheets/RAG_Security_Cheat_Sheet.md
11. https://cheatsheetseries.owasp.org/cheatsheets/RAG_Security_Cheat_Sheet.html
12. https://www.cloudflare.com/learning/ai/owasp-top-10-risks-for-llms/
13. https://learn.microsoft.com/fr-fr/training/modules/owasp-top-10-for-dotnet-developers/12-server-side-request-forgery
14. https://wstg.owasp.org/latest/4-Web_Application_Security_Testing/07-Injection/19-Server-Side_Request_Forgery/
15. https://truto.one/blog/how-to-maintain-document-level-rbac-in-enterprise-rag-pipelines/